What Decides the Reported Accuracy?
An Empirical Study of LLM-Based Triton GPU Kernel Generation

Anonymous ACL submission

Abstract

Many recent papers use large language models to write GPU kernels, and they report accuracy on the same benchmark. The reported numbers, however, are very far apart. Some papers report almost zero, and other papers report one hundred percent. This paper does not propose a new generation system. Instead, we ask what causes this gap. We measure 8 commercial models and 4 open models on three GPUs and on four benchmarks, and we use one fixed evaluation harness for every run. We also take released artifacts from earlier work, such as a Triton-specific reinforcement learning model, an agent framework, a failure-rule memory, and profiling feedback, and we run them in our own setting. We report eight findings. A large part of the gap does not come from the model at all. It comes from the evaluation setup and from hidden limits in the serving stack. One such defect alone moves accuracy from 0.45 to 0.75. Most of the rest of the gap comes from the test-time compute budget. The same free 8B open model reaches 0.44 under a small budget and 0.95 under a large budget. A larger budget, however, only helps inside the training distribution. Outside it, and on a new domain-specific language, a larger budget changes nothing. We also find that breadth beats depth, and that domain specialization beats model size. We close with the six axes that a paper should report together with its accuracy number.

1 Introduction

GPU kernels decide much of the cost of deep learning systems. Writing a fast kernel by hand is hard, because the programmer must reason about the memory hierarchy, about thread synchronization, and about warp-level behavior (Kirk and Hwu, 2016). OpenAI Triton (Tillet et al., 2019) lowers this barrier. It offers a tile-based programming model in Python and leaves many low-level choices to the compiler. Triton became the default backend of `torch.compile` in PyTorch 2.0 (Ansel et al., 2024), so it is now a natural target for automatic code generation. Many groups therefore try to generate Triton and CUDA kernels with large language models, and they measure the result on KernelBench (Ouyang et al., 2025), which has become the standard benchmark.

The problem is that the reported numbers do not agree. KernelEvolve reports one hundred percent (Liao et al., 2025). CudaForge reports 97.6 percent (Zhang et al., 2025b). QiMeng reports near-perfect accuracy on the first two levels (Zhu et al., 2026). At the same time, other studies on the same benchmark report single-digit accuracy. A reader cannot tell whether this gap comes from model ability, from search budget, or from the measurement setup itself. Each paper uses a different number of attempts, a different verification strength, and a different serving stack, and most papers do not report these settings at all.

This paper is therefore not a system paper. We do not propose a new generator, a new agent, or a new training method. We audit the current state of the field. We fix one evaluation harness and then change one axis at a time on top of it. We vary the model, the GPU, the benchmark, and the compute budget. We also take artifacts that earlier work has released, including a Triton-specific reinforcement learning model (Li et al., 2025b), a multi-turn reinforcement learning model together with its training environment (Liu et al., 2026), an open deep-agent framework, a failure-rule memory (Du et al., 2026), and profiling feedback (Zhang et al., 2025b). We run each of them in our own setting and we report where the reported benefit holds and where it breaks. We organize the study around five questions. RQ1 asks how accurate current models really are under one common setting. RQ2 asks how much the measurement setup alone decides the reported number. RQ3 asks what the failures look like and how they change as models improve. RQ4 asks what actually raises accuracy, that is, depth

of repair, breadth of sampling, test-time budget, or reinforcement learning. RQ5 asks whether the artifacts of earlier work transfer to another harness and to another distribution.

We organized more than one hundred related papers into eight groups, namely CUDA generation, Triton generation, reinforcement learning, agent and evolutionary frameworks, compiler and IR level work, benchmarks, evaluation methodology, and supervised foundation models. Two gaps became visible. First, many papers propose a single technique, but we did not find a paper that compares several models, several GPUs, and several benchmarks under one controlled harness. Second, papers that report perfect accuracy always assume a large attempt budget, such as unbounded tree search, repeated repair with profiling, or large-scale reinforcement learning, yet they do not report the single-attempt number next to it. The individual parts that we use, such as iterative repair, best-of-N sampling, model cascading, and failure-rule reuse, are all known. What is new here is the controlled measurement on top of them.

Our contribution is a set of findings rather than a new method, and we state them here so that this section alone is enough to see the result of the study. First, the newest commercial flagship models converge to almost the same behavior below a certain difficulty, and a free 8B Triton-specific open model reaches the same level, so specialization beats size. Second, changing only the evaluation infrastructure and the hidden serving limits, and not the model, moves accuracy by up to 0.30, so accuracy numbers are not comparable across papers unless these settings are stated. Third, as models get better, the dominant failure moves from compilation errors to value mismatches, which also explains why profiling feedback does not help before correctness is solved. Fourth, independent samples help more than deeper repair of a single sample. Fifth, the gap between our full-benchmark numbers and the perfect scores in the literature is mostly a difference of budget regimes, and reinforcement learning and budget are complements rather than substitutes. Sixth, an agent framework improves search efficiency but does not raise the ceiling of the base model, so a strong open model with an open framework can reach a perfect score without any commercial model. Seventh, aggressive quantization destroys quality on harder tasks, while full precision on a very large model destroys throughput, so a mid-size model at full precision is the practical choice. Eighth, and most important as a limit of the whole picture, a larger budget only helps inside the training distribution. Out-of-distribution tasks and a new domain-specific language do not improve at all. From these findings we derive six axes that future papers should report next to accuracy.

2 Related Work

Benchmarks and learning. KernelBench (Ouyang et al., 2025) defines 250 tasks in three levels and asks the model to turn a PyTorch reference into an equivalent GPU kernel. TritonBench (Li et al., 2025a) collects real Triton operators from GitHub, and TritonGym (Guan et al., 2026) standardizes tool access so that model ability and workflow design can be measured apart from each other. Several surveys track this fast-moving area (Yu et al., 2026; Zhang et al., 2026; Gong et al., 2025). On the learning side, AutoTriton (Li et al., 2025b) combines supervised training and GRPO and reports commercial-level quality from an 8B model. TritonRL (Woo et al., 2025) uses verifiable rewards to suppress reward hacking, and DRTriton (Guo et al., 2026) uses large synthetic data. For CUDA, CUDA-L1 (Li et al., 2025c) uses contrastive reinforcement learning and Kevin (Baronio et al., 2025) uses multi-turn reinforcement learning. Tehrani et al. (2026) fine-tune a frontier model and raise single-attempt accuracy. Dr. Kernel (Liu et al., 2026) names two failure modes of kernel reinforcement learning, namely reward hacking and lazy optimization, and releases both a distributed environment and a multi-turn advantage estimator. CUDA Agent (Dai et al., 2026) and MusaCoder (Cheng et al., 2026) extend this line to large agentic training and to a new accelerator. All of these methods need data curation and a large amount of compute. We do not train anything. We download what these papers release, we run it in our harness, and we ask whether the reported benefit survives a change of harness and a change of distribution.

Agent frameworks. A second line uses a fixed model inside a repair loop and adds no training. GEAK (Wang et al., 2025) uses a generator, a reflector, an evaluator, and an optimizer. CudaForge (Zhang et al., 2025b) adds hardware profiling metrics. EGG (Han et al., 2026) splits kernel generation into algorithm design and hardware tuning, and KernelBand (Ran et al., 2026) treats optimization as a bandit problem. STARK (Dong

et al., 2025) separates planning, coding, and debugging, and PRAGMA (Lei et al., 2025) turns profiling data into explicit reasoning steps. AccelOpt (Zhang et al., 2025a) reuses an experience memory, and KernelEvolve (Liao et al., 2025) runs tree search across several hardware platforms. AdaExplore (Du et al., 2026) distills repeated failures into reusable rules and keeps search diverse. KForge (Sereda et al., 2026) and Kernel Foundry (Huang et al., 2026) alternate a correctness pass and an optimization pass. Other work reduces the reasoning load by exposing a compact language instead of raw CUDA, such as μCUTLASS (Hari et al., 2026), CuTeGen (Saba et al., 2026), and QiMeng-Kernel (Zhu et al., 2026), or by adding hardware context, such as SwizzlePerf (Tschand et al., 2025). LEGO (Xu et al., 2026) moves from single kernels to whole model graphs. These frameworks all assume a large attempt budget, and this is exactly the axis that we control.

Verification and evaluation. A generated kernel can compile and still be wrong, so verification is now a topic of its own. Kwon et al. (2026) build a verification chain between the compiler and the runtime. COLT (Tang et al., 2026) lets several small models cooperate and escalates to a larger model when needed, and Crupi et al. (2026) train a small model as a judge to pick a correct candidate cheaply. AutoMegaKernel (Jaber and Jaber, 2026) rejects wrong proposals before execution with a static checker, and AgentCompile (Li et al., 2026) keeps the correctness decision inside the compiler. The risk of an unverified number is real, because inflated results from reward hacking have already been reported (Lange et al., 2025). We extend this concern from verification to the measurement setup as a whole.

3 Study Design

Figure 1 shows the structure of the study. Every run shares one harness. The harness builds the prompt, calls the model, extracts the code from the answer, scores the kernel inside an isolated subprocess, classifies the error, and then repairs and retries. The isolation step is not an optimization. The CUDA runtime keeps state per process, so a single bad kernel can corrupt the context and make every later evaluation in the same process fail. Without isolation a sequential run over 250 tasks is not meaningful. For correctness we draw two random inputs and require an absolute error below 10^{-2} against the reference on both. For speed we take the median of ten timed runs and report Fast-0, Fast-1, and Fast-2 as defined by the benchmark (Ouyang et al., 2025).

We change four axes on top of this fixed harness. The first axis is the model. We use 8 commercial models, namely gpt-4o-mini, gpt-4o, gpt-5.2, gpt-5.2-pro, gpt-5.4-nano, gpt-5.5, gemini-3.1-pro-preview, and claude-opus-4-7. We also use 4 open models, namely Qwen3-Coder-30B-A3B (Hui et al., 2024), AutoTriton-8B (Li et al., 2025b), a quantized GLM-5.2 in a 2-bit and a 4-bit version, and Dr. Kernel-14B (Liu et al., 2026). The second axis is the hardware. We use an RTX PRO 6000 with 96GB, an H100 with 80GB, and an A6000 with 48GB. The third axis is the benchmark. We use KernelBench with 250 tasks, TritonBench with 166 tasks, TritonGym with 139 standard tasks and 13 out-of-distribution tasks, and FlashInfer-Bench with 8 dense tasks (FlashInfer Team, 2025). The fourth axis is the compute budget. We vary the number of samples, the number of repair rounds, and the number of effective turns in a multi-turn setting from 1 up to 14. Commercial models run through a unified gateway. Open models run on a local vLLM server, or on llama.cpp for the quantized model.

The software stack is Triton 3.5.1 on the H100 and Triton 3.6 on the RTX PRO 6000, with PyTorch 2.x, Python 3.12, and CUDA 12.x. We call a budget *small* when we take one sample and up to three repair rounds, and we call it *large* when we take three samples and up to nine repair rounds, which is at most 27 attempts per task.

4 Results

4.1 RQ1: How Accurate Are Current Models?

We first fix the small budget and run every model on all 100 tasks of KernelBench Level 1. Table 1 reports the result; runs that covered only a subset of the tasks are excluded.

The effect of model generation is clear. Accuracy grows from 19 percent for gpt-4o to between 43 and 46 percent for the newest flagship models. The last row is the more interesting one. A free open model with only 8B parameters, trained specifically for Triton, reaches the level of the best commercial flagship. We repeated this run four times with the same harness and budget and obtained 0.47, 0.44, 0.39, and 0.47, so one run of this

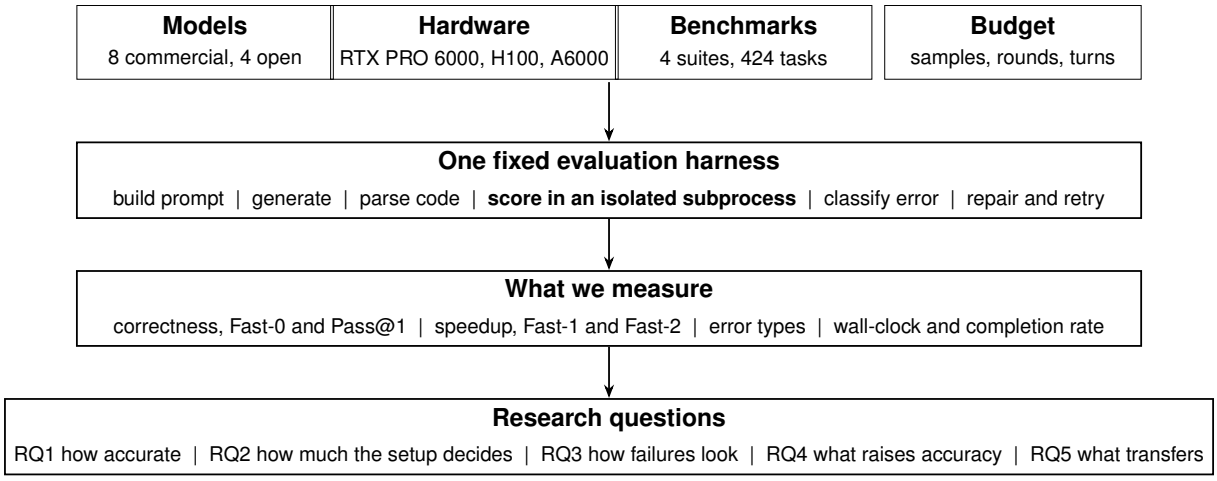


Figure 1: Structure of the study. We keep one evaluation harness fixed and change four axes on top of it. The harness scores every kernel in an isolated subprocess. This is required, not optional, because one bad kernel can corrupt the CUDA context, and then every later evaluation in the same process fails. We do not propose a new generation system. We hold the harness constant and change one axis at a time, so that we can tell apart the effect of the model, the hardware, the benchmark, and the compute budget.

Table 1: Accuracy on all 100 tasks of KernelBench Level 1 under the small budget. Every row uses the same harness and the same task set. The AutoTriton row is the mean and standard deviation over four independent runs; every other row is a single run.

Model	Type	Fast-0
gpt-4o-mini	commercial	15%
gpt-4o	commercial	19%
Qwen3-Coder-30B	open, 30B	27%
gpt-5.4-nano	commercial	35%
gpt-5.5	commercial	37%
gemini-3.1-pro-preview	commercial	43%
claude-opus-4-7	commercial flagship	46%
AutoTriton-8B	open, 8B, RL	$44\% \pm 4$

benchmark carries a standard deviation of about four points. Differences of that size, such as the two points between the 8B model and the best flagship, are ties; the gaps between model generations are an order of magnitude larger. The four runs solved 59 distinct tasks but only 25 in common (see Finding 4). We also observe that gpt-5.5 and claude-opus-4-7 solve exactly the same 37 tasks among the first 44 and only separate on the last part of the benchmark, and gemini-3.1-pro-preview shows the same late recovery, so this looks like a property of the flagship generation rather than of one model.

Finding 1. *Recent commercial flagship models converge to nearly the same behavior below a certain difficulty, and a free 8B Triton-specific open model reaches that same level. For kernel generation, domain specialization beats model size.*

These absolute numbers, between 15 and 46 percent, are far below the perfect scores in the literature. The next two subsections show that most of this gap comes from axes other than model ability.

4.2 RQ2: How Much Does the Setup Decide?

While auditing our own pipeline we found a series of defects that lower accuracy without any relation to model ability. Table 2 lists them together with the recovery after the fix.

The first row is the clearest case. When the serving process holds about 90 percent of the GPU, the scorer runs out of memory on tasks with large inputs. One Level 1 task on ReLU has a single input tensor of 6.4GB. As a result the harness recorded a failure even for ReLU, which is semantically trivial. Lowering the serving share and freeing tensors after every trial raised accuracy from 0.45 to 0.75 with no change to the model. The other rows have the same character. The harness counts them as tasks that the model failed to solve, but in fact the code was cut before it arrived, or the scorer broke down on its own.

Three of these are hidden variables. The response token cap of the framework, the generation cap inside the released model artifact, and the serving context length never appear in any accuracy report. A reasoning model spends thousands of to-

Table 2: Defects in the measurement setup and the recovery after the fix. In every row the model stays the same and only the environment changes.

Defect	Symptom	Fix	Accuracy
GPU memory contention	serving holds 90% of the GPU, so large inputs fail with out of memory	lower the serving share and free tensors per trial	0.45 to 0.75
No input dtype contract	the model assumes fp16 while the reference is fp32	state the dtype contract in the prompt	0.65 to 1.00
No gating, no context reset	a polluted context blocks every repair round	gate on correctness and restart from a clean context	
Framework token cap	the reasoning trace is cut before the code block	raise the response token limit	0.37 to 0.78
Artifact generation cap	the served limit clamps the client limit	fix the generation config of the artifact	truncation gone
Serving context length	the repair prompt overflows and every round is lost	enlarge the serving context	failure gone

kens on its thinking trace before it writes any code, so a common 8K cap turns a solvable task into an empty answer.

Finding 2. *Changing only the evaluation infrastructure and the hidden serving limits, without touching the model, moves accuracy by as much as 0.30. Accuracy numbers from different systems are therefore not comparable unless these settings are reported.*

Our own code base gives a second example. Two evaluation paths that share model, prompt, and budget but differ in thread count, timeout, temperature, and verification inputs read 0.44 $\pm$ 0.04 and 0.35 $\pm$ 0.05 for the same 8B model (Appendix M).

We also ported the failure-rule memory of earlier work (Du et al., 2026), which distills each failure into one short rule and reuses it across tasks. The rules that the system collected while the memory defect was still active were wrong, for example a rule that said the GPU does not have enough memory to run the kernel. These wrong rules had high weight and filled the prompt, and the model then failed even on a plain square matrix multiplication. Accuracy recovered after we cleared the store. Experience reuse is therefore only an asset when the evaluation environment is already healthy, and a shared store spreads the damage across servers.

4.3 RQ3: What Do the Failures Look Like?

We classify each failure as a compilation error, a value mismatch, a CUDA error, or a timeout. Table 3 reports the distribution.

The dominant error moves in a regular way as the model gets better. Compilation errors fall from 71.8 percent to 0 percent, while value mismatches rise from 20 percent to 65 percent. Learning the syntax of Triton is comparatively easy for a language model, but understanding the numerical meaning of a GPU operation stays hard.

Table 3: Share of each error type among failed tasks on KernelBench Level 1. The timeout column counts our own response and evaluation time limits together with unclassified runtime errors such as API rate limits. It is not a constraint of the benchmark.

Model	Compile	Value	CUDA	Timeout
gpt-4o-mini	71.8%	20.0%	2.4%	2.4%
gpt-4o	45.7%	37.0%	11.1%	0%
gpt-5.2	6.9%	58.6%	1.7%	20.7%
claude-opus-4-7	0%	65%	0%	28%

This diagnosis has a practical consequence. We ran an A/B test on three models in which we injected Nsight Compute hardware counters into the optimization prompt. Accuracy did not improve for any model, and the speedup gain was very small. A profiling signal only applies to a kernel that is already correct, while the real bottleneck is correctness. Profiling-driven optimization therefore only makes sense after correctness is solved.

Finding 3. *As models improve, failures move from syntactic errors to semantic errors. Profiling feedback has no effect before correctness is solved, so a correctness-first and profiling-later design is required.*

The timeout column needs a separate reading. For the reasoning model gpt-5.2-pro, 100 percent of the Level 1 failures were timeouts. The long thinking trace did not fit inside our response time limit, so this is an operational constraint of the setup and not a limit of ability.

Table 4: Effect of the number of repair rounds for gpt-4o on KernelBench Level 1.

Rounds	Solved	Compile err.	Value mism.
1	19	59.3%	24.7%
3	21	51.9%	39.2%
5	19	45.7%	37.0%
10	22	32.1%	53.8%
Union	**28**	—	—

4.4 RQ4: What Actually Raises Accuracy?

4.4.1 Depth versus breadth.

We ran gpt-4o on Level 1 with 1, 3, 5, and 10 repair rounds. The individual runs solved 19, 21, 19, and 22 tasks, so deeper repair alone barely helped. The union of the four runs, however, solved 28 tasks, which is 27.3 percent above the best single run. Each run therefore solves some tasks that the other runs miss. The four identical AutoTriton runs of Section 4.1 show the same effect with no change of budget: their union covers 59 tasks against 47 for the best run. Table 4 also shows that the error profile shifts with depth. More rounds remove compilation errors but leave value mismatches behind, which matches Finding 3.

The same effect holds across models. In a cascade, gpt-4o alone solved 25 tasks, and adding the next generation model raised the total to 61 tasks, which is a factor of 2.44. The union of five models reached 51 out of 100 and beat the best single model at 46 out of 100 by 10.9 percent. The gain was not uniform. Adding an expensive reasoning model as the last stage added only one task out of 225, so its cost was not justified. We also note that one recent flagship model alone, at 46 out of 100, already beats the earlier three-model cascade at 43 out of 100, so a better model generation reduces the need for a multi-model strategy.

Finding 4. *Breadth beats depth. Given the same budget, several independent attempts are more useful than deeper repair of one attempt.*

4.4.2 Test-time budget.

When we change only the budget axis, most of the gap disappears. Table 5 shows that the free 8B model moves from 0.44 under the small budget to 0.95 under the large budget, which is close to the perfect scores in the literature.

A larger budget, however, is limited by wall-clock time. In the full run with the large budget, 27 attempts per task did not fit into a two-hour limit on a slow local server, and 59 percent of the tasks did not finish. The real bottleneck of test-time scaling is therefore not the number of attempts but the throughput per attempt, which is why papers that report perfect accuracy assume a fast commercial API or a large serving cluster.

Table 5: Level 1 accuracy when only the budget changes. Small budget is one sample and three rounds. Large budget is three samples and nine rounds. The value marked with a dagger is computed over the 38 tasks that finished inside the time limit. The AT-8B small-budget value is the mean of four runs (Section 4.1); all other values are single runs.

Budget	Scope	Qwen-30B	AT-8B	opus
small	all 100	0.27	0.44	—
large	first 20	0.75	**0.95**	**1.00**
large	all 100	—	0.76†	—

Table 6: Fast@1.2 in percent as a function of the number of effective turns. Only the turn count changes. DrK-14B is trained with kernel-specific reinforcement learning and Qwen3-30B is not.

Model and level	1 turn	3 turns	7 turns	14 turns
Qwen3-30B, L2	3.7	11.9	16.8	28.4
DrK-14B, L2	17.4	41.3	60.6	**76.9**
DrK-14B, L1	11.5	22.0	27.0	31.2
DrK-14B, L3	3.0	8.3	13.0	20.5

4.4.3 Reinforcement learning and budget.

We reproduced a released multi-turn reinforcement learning environment and changed only the number of effective turns from 1 to 14. Table 6 compares a model without such training and a model with it.

Both models improve monotonically with the budget, so budget dominance is a general effect and not a property of one model. The trained model, however, raises both the intercept, by a factor of 4.7 at one turn, and the gain per turn, by 59.5 points against 24.7 points. The saturation point also depends on difficulty. Level 1 saturates around seven turns, while Level 3 has not saturated even at fourteen turns and shows the largest relative gain.

Finding 5. *The perfect scores in the literature and our full-benchmark numbers are measurements of different budget regimes, not of different model ability. Reinforcement learning and test-time budget are complements, and the saturation point depends on task difficulty.*

4.5 RQ5: What Transfers to Another Setting?

4.5.1 Reusing a failure-rule memory.

Beyond the poisoning case in Section 4.2, the failure-rule memory is useful when the harness is healthy. The system turns each failure into one short rule, for example a rule that says the kernel must not call PyTorch operations inside the Triton kernel body, and it keeps these rules across tasks. During one full evaluation the store collected between 73 and 80 distinct rules, and the rules covered common mistakes such as wrong tile dimensions, a wrong block size along the reduction axis, and missing contiguity assumptions. This kind of memory is cheap, since it only needs one extra model call per failure, and it is the most portable of all the artifacts that we reused. Its risk, however, is asymmetric. A useful rule saves a few attempts, while a wrong rule that carries a high weight can block a whole class of tasks, as we saw when accuracy collapsed on tasks as simple as a square matrix multiplication.

4.5.2 Reproducing a released training environment.

We first downloaded the official code and the released weights of a kernel-specific reinforcement learning study and checked whether the reported numbers reproduce. On the 100 tasks of Level 2 we measured Fast@1.2 at 24.3 percent against 25.6 percent in the paper, and our Fast@1.5 and Fast@2 were even higher. The reported result is therefore real. Reproduction, however, needed an eight-GPU node. On a single GPU the trainer and the serving engine did not fit together, so reproducibility does not imply accessibility.

4.5.3 Cross-hardware and cross-benchmark results.

Table 7 reports two open models on two GPUs across all levels. The advantage of the 8B Triton-specific model is consistent, and the difficulty cliff at Level 2 and Level 3 is sharp. Accuracy falls from 0.44 on single operations to 0.10 on fused operations and to 0.06 on whole models. On the real serving kernels of FlashInfer-Bench only one combination reaches 0.25 while the others stay at zero, which points to differences in Triton version and compute architecture.

Accuracy itself is almost independent of the GPU as long as memory is sufficient. Under the same large budget on the first 20 tasks, the 30B open model reached 0.75 on the RTX PRO 6000 and 0.60 on the H100, and the whole difference came from memory. The H100 has 80GB, so after serving the 30B model the scorer only had about 43GB, while the RTX PRO 6000 with 96GB left about 52GB. After we correct for the out-of-memory failures the H100 number rises to about 0.70. A third GPU, the A6000 with 48GB, gave a similar profile across levels, with 0.37 on Level 1, 0.12 on Level 2, and 0.04 on Level 3, which supports the same conclusion.

Table 7: Open models on two GPUs across all levels under the small budget. PRO 6000 is the RTX PRO 6000 and AT-8B is the Triton-specific model. On the H100 the 30B model leaves too little memory for the scorer, so the value marked with a dagger is depressed by out-of-memory failures. See Finding 2. The AT-8B value at Level 1 on the PRO 6000 is the mean of four runs; all other values are single runs.

Level	Qwen-30B		AT-8B	
	PRO 6000	H100	PRO 6000	H100
L1, 100 tasks	0.27	0.12†	**0.44**	0.43
L2, 100 tasks	0.05	0.06	**0.10**	0.08
L3, 50 tasks	0.04	0.06	**0.06**	0.02
FlashInfer, 8	0.00	0.00	0.00	**0.25**

Speedup behaves in the opposite way. On a square matrix multiplication we measured 1.61 times on the RTX PRO 6000 and 0.83 times on the H100, because the cuBLAS baseline of the H100 is very strong. A speedup number is therefore only comparable inside one hardware setting. The quality of the optimization also improves with model generation. Among correct kernels, the share that reaches at least a two times speedup grows from 6.3 percent for gpt-4o-mini and 16.0 percent for gpt-4o to 28.6 percent for gpt-5.4-nano and 30.4 percent for claude-opus-4-7. The largest single speedup that we observed was 84.03 times on one Level 3 task.

4.5.4 Agent structure against model ability.

We downloaded an open deep-agent framework that reports perfect accuracy and ran it with our own open models. With Qwen-30B it solved 15 of the first 20 tasks, which is exactly the same score that our own pipeline reaches under the large budget, and even the set of failed tasks was identical. The wall-clock time, however, dropped from several hours to about one hour. With a stronger open model, namely GLM-5.2 in 4-bit form, the same framework solved all 20 tasks and matched the

commercial flagship.

Finding 6. *An agent framework improves search efficiency but does not raise the ceiling of the base model. Accuracy is a product of framework efficiency and model ability, and a strong open model with an open framework can reach a perfect score without any commercial model.*

Extending this combination to all of Level 1 gave 0.48 over the 77 tasks that we ran. Successes concentrated on matrix multiplication, reduction, and element-wise tasks, while convolution and sequential scan tasks dropped sharply. To separate budget from ability we removed truncation completely, with a 32K serving context and a response limit of 24000 tokens, and retried three representative convolution tasks. Truncation disappeared, yet all three still failed to converge. The remaining convolution failures are therefore a limit of model ability on indexing large tensors, not a limit of the token budget.

4.5.5 Model size against weight precision.

We loaded a very large open reasoning model, GLM-5.2, on an eight-GPU node with layer splitting. In 2-bit form it reached only 0.10 on Level 1 and solved nothing at all on Level 2, Level 3, and the serving kernels, after 21.5 hours of compute. A general reasoning model with two orders of magnitude more parameters therefore stayed below half of the 8B specialized model in Table 7. To confirm that quantization was the cause, we reloaded the same model in 4-bit form. Accuracy recovered to 0.96 over the tasks that finished inside the time limit, which is the best result among our open models. This version, however, generated only about 8 tokens per second, so 76 percent of the tasks exceeded the limit. The bottleneck simply moved from precision to throughput.

Finding 7. *Quantization loss grows with task complexity. Low precision destroys quality and full precision on a very large model destroys throughput, so a mid-size model at full precision is the practical choice under a fixed budget.*

4.5.6 The boundary of budget.

All results above hold inside the training distribution. To find the boundary we moved a kernel-specific reinforcement learning model into a standardized agent environment and compared a single attempt against a test-time scaling agent. On the 139 standard tasks Pass@1 rose from 12.2 percent to 18.0 percent, and the performance of the passing kernels relative to the oracle rose from 0.143 to 0.285. On the 13 out-of-distribution tasks, however, Pass@1 stayed at 7.7 percent and did not move at all. On a new domain-specific language the same model solved none of the three tasks that our environment could run. Appendix J repeats the test on two NPU targets. The same pattern appears across benchmarks. On the Triton-specific benchmark the advantage of the specialized model disappears and all four combinations converge into a narrow band between 0.15 and 0.19.

To separate the cause we ran a two-by-two ablation over the prompt format and the budget. Changing only the prompt format had almost no effect, and execution accuracy even fell from 0.151 to 0.127. Turning on the budget, in contrast, raised execution accuracy by about 2.3 times and call accuracy by about 2.5 times. The gap against the original paper is therefore explained by single-attempt evaluation against repeated repair, and not by a mismatch with the training prompt. This repeats what we found on KernelBench, so budget dominance is a general effect and not a property of one benchmark.

Finding 8. *A larger test-time budget only improves accuracy and performance inside the distribution. It does not rescue the collapse outside the distribution. The benefit of reinforcement learning is also tied to its own training harness and transfers only in part.*

5 Conclusion

This paper audits the current state of LLM-based Triton kernel generation with five research questions instead of proposing a new system. We measured 8 commercial models and 4 open models on three GPUs and on four benchmarks with one fixed harness, and we ran released artifacts of earlier work in our own setting. The reported accuracy of a system is not a function of model ability alone. It is a function of ability, of test-time budget, and of the measurement setup. Removing infrastructure defects, giving a sufficient budget, and using a specialized open model is enough to reach the level of the best reported results without any commercial model. This recipe, however, only works inside the training distribution, and no budget helps outside it. We therefore ask that future work report six axes next to any accuracy number: the attempt budget, the verification strength, the memory headroom of the scorer, the serving context length, the response token caps of both the framework and the model artifact, and the evaluation distribution.

Limitations

Only one configuration, the 8B open model under the small budget on Level 1, was run more than once. Its run-to-run standard deviation is about four points (Section 4.1), and every other number in this paper comes from a single run, so differences of a few points between the rows of a table should not be read as a ranking. The absolute values are also tied to the harness settings that we list, as the two evaluation paths of Section 4.2 show. The two NPU experiments in Appendix J cover five operators and three kernels respectively, and the second of them was run with one open model only, so they are evidence of a pattern and not a benchmark. Some of our large-budget measurements ran on the first 20 tasks of Level 1 rather than on all 100 tasks, and the full run left unfinished tasks because of the wall-clock limit. Commercial models sit behind an API, so we cannot control their serving conditions, and an uncontrolled infrastructure difference remains in every comparison between commercial and open models. The out-of-distribution set has only 13 tasks, so it shows that a boundary exists but does not measure its size precisely. Finally, our timeout column mixes true time limits with unclassified runtime errors, so it should be read as an operational category rather than as a single failure cause.

Ethical Considerations

This study does not involve human subjects, personal data, or annotators. All benchmarks and model artifacts that we reuse are publicly released by their authors for research use, and we cite each of them. The KernelBench code, TritonGym, AdaExplore, and the GLM-5.2 weights are released under the MIT license; TritonBench, FlashInfer-Bench, the Qwen3-Coder weights, and the AutoTriton weights under Apache-2.0; the Dr. Kernel training code states Apache-2.0 in its documentation. The KernelBench task data and the Dr. Kernel weights carry no explicit license, and we use them only for evaluation as their authors intend. The two NPU software stacks of Appendix J are vendor SDKs used under their own terms on hardware available to us. We do not redistribute any weights or data, and we release no new artifact with this submission. The main risk of the work is misuse of generated kernels in production without verification, which we address by making verification strength one of the six reporting axes. We report the wall-clock cost of each experiment in Appendix B so that the compute can be weighed against the findings.

References

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. 2024. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *ASPLOS*.

Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. 2025. Kevin: Multi-turn rl for generating cuda kernels. *Preprint*, arXiv:2507.11948.

Kun Cheng, Songshuo Lu, Sicong Liao, Tankun Li, Yafei Zhang, et al. 2026. MusaCoder: Native GPU kernel generation with full-stack training on Moore Threads GPUs. *Preprint*, arXiv:2606.04847.

Giuseppe Crupi, Rosalia Tufano, and Gabriele Bavota. 2026. Improving code generation via small language model-as-a-judge. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*.

Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, et al. 2026. CUDA Agent: Large-scale agentic reinforcement learning for high-performance CUDA kernel generation. *Preprint*, arXiv:2602.24286.

Juncheng Dong, Yang Yang, Tao Liu, Yang Wang, Feng Qi, Vahid Tarokh, Kaushik Rangadurai, and Shuang Yang. 2025. Stark: Strategic team of agents for refining kernels. *Preprint*, arXiv:2510.16996.

Weihua Du, Jingming Zhuo, Yixin Dong, Andre Wang He, Weiwei Sun, Zeyu Zheng, Manupa Karunaratne, Ivan Fox, Tim Dettmers, Tianqi Chen, Yiming Yang, and Sean Welleck. 2026. Adaexplore: Failure-driven adaptation and diversity-preserving search for efficient kernel generation. *Preprint*, arXiv:2604.16625.

FlashInfer Team. 2025. FlashInfer-Bench: Building the virtuous cycle for AI-driven LLM systems. `https://github.com/flashinfer-ai/flashinfer-bench`. Benchmark suite of production LLM serving kernels; blogpost at `https://flashinfer.ai/2025/10/21/flashinfer-bench.html`.

Jingzhi Gong, Vardan Voskanyan, Paul Brookes, Fan Wu, Wei Jie, Jie Xu, Rafail Giavrimis, Mike Basios, Leslie Kanthan, and Zheng Wang. 2025. Language models for code optimization: Survey, challenges and future directions. *Preprint*, arXiv:2501.01277.

Yue Guan, Yichen Lin, Xu Zhao, Jianzhu Yao, Xinwei Qiang, Zhongkai Yu, Pramod Viswanath, Yufei Ding, and Adnan Aziz. 2026. TritonGym: A benchmark for

agentic LLM workflows in Triton GPU code generation. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. Code and leaderboard: https://github.com/yil384/TritonGym.

Siqi Guo, Ming Lin, and Tianbao Yang. 2026. Dr-triton: Large-scale synthetic data driven reinforcement learning for triton kernel generation. *Preprint*, arXiv:2603.21465.

Yaochen Han, Ke Fan, Hongxu Jiang, Wanqi Xu, Weiyu Xie, Runhua Zhang, Chenhui Zhu, and Yixiang Zhang. 2026. EGG: An expert-guided agent framework for kernel generation. *Preprint*, arXiv:2606.26758.

Siva Kumar Sastry Hari, Vignesh Balaji, Sana Damani, Qijing Huang, and Christos Kozyrakis. 2026. Improving efficiency of GPU kernel optimization agents using a domain-specific language and speed-of-light guidance. *Preprint*, arXiv:2603.29010.

Zixuan Huang, Da Chen, Kecheng Huang, Lihao Yin, Xing Li, Huiling Zhen, Mingxuan Yuan, and Zili Shao. 2026. Kernel Foundry: A diagnosis-driven evolutionary kernel optimizer with multi-experts. *Preprint*, arXiv:2605.30359.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*.

Jaber Jaber and Osama Jaber. 2026. AutoMegaKernel: A statically-checked agent harness for self-retargeting megakernel synthesis. *Preprint*, arXiv:2606.09682.

David B. Kirk and Wen-mei W. Hwu. 2016. *Programming Massively Parallel Processors: A Hands-on Approach*, 3rd edition. Morgan Kaufmann.

Hyunho Kwon, Sanggyu Shin, Ju Min Lee, Hoyun Youm, Seungbin Song, Seongho Kim, Hanwoong Jung, Seungwon Lee, and Hanjun Kim. 2026. Compiler-runtime co-operative chain of verification for LLM-based code optimization. In *Proceedings of the 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*.

Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. 2025. The AI CUDA engineer: Agentic CUDA kernel discovery, optimization and composition. Technical report, Sakana AI.

Kelun Lei, Hailong Yang, Huaitao Zhang, Xin You, Kaige Zhang, Zhongzhi Luan, Yi Liu, and Depei Qian. 2025. Pragma: A profiling-reasoned multi-agent framework for automatic kernel optimization. *Preprint*, arXiv:2511.06345.

Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025a. Tritonbench: Benchmarking large language model capabilities for generating triton operators. In *Findings of the Association for Computational Linguistics (ACL)*.

Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025b. AutoTriton: Automatic triton programming with reinforcement learning in LLMs. *arXiv preprint arXiv:2507.05687*.

Xiaoya Li, Albert Wang, Guoyin Wang, Jiwei Li, and Chris Shum. 2025c. Cuda-l1: Improving cuda optimization via contrastive reinforcement learning. *Preprint*, arXiv:2507.14111.

Xuanzhe Li, Ziyan Weng, Zhiyu Zhu, and Junhui Hou. 2026. AgentCompile: An LLM-guided compiler for direct CUDA inference. *Preprint*, arXiv:2606.07665.

Gang Liao, Carole-Jean Wu, Gaoxiang Liu, et al. 2025. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta. *Preprint*, arXiv:2512.23236.

Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. 2026. Dr. Kernel: Reinforcement learning done right for Triton kernel generations. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. KernelGYM environment and code available at https://github.com/hkust-nlp/KernelGYM.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. ArXiv:2502.10517.

Dezhi Ran, Shuxiao Xie, Mingfang Ji, Anmin Liu, Mengzhou Wu, Yuan Cao, Yuzhe Guo, Hao Yu, Linyi Li, Yitao Hu, Wei Yang, and Tao Xie. 2026. KernelBand: Steering LLM-based kernel optimization via hardware-aware multi-armed bandits. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. Code available at https://github.com/TongmingLAIC/KernelBand.

Tara Saba, Zhiyang Chen, Jikai Jason Li, Anne Ouyang, Xujie Si, and Fan Long. 2026. Cutegen: An llm-based agentic framework for generation and optimization of high-performance gpu kernels using cute. *Preprint*, arXiv:2604.01489.

Taras Sereda, Burak Bartan, Ankita Nayak, Tom St. John, Natalie Serrino, and Zain Asgar. 2026. KForge: LLM-driven cross-platform kernel generation for AI accelerators. In *Machine Learning for Architecture and Systems Workshop (MLArchSys), co-located with ISCA*.

Annabelle Sujun Tang, Christopher Priebe, Lianhui Qin, and Hadi Esmaeilzadeh. 2026. COLT: Lightweight multi-LLM collaboration through shared MCTS reasoning for model compilation. *Preprint*, arXiv:2602.01935.

Ali Tehrani, Yahya Emara, Essam Wissam, Wojciech Paluch, Waleed Atallah, Łukasz Dudziak, and Mohamed S. Abdelfattah. 2026. Fine-tuning gpt-5 for gpu kernel generation. *Preprint*, arXiv:2602.11000.

Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An intermediate language and compiler for tiled neural network computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19.

Arya Tschand, Muhammad Awad, Ryan Swann, Kesavan Ramakrishnan, Jeffrey Ma, Keith Lowery, Ganesh Dasika, and Vijay Janapa Reddi. 2025. SwizzlePerf: Hardware-aware LLMs for GPU kernel performance optimization. *Preprint*, arXiv:2508.20258.

Jianghui Wang, Vinay Joshi, Saptarshi Majumder, Xu Chao, Bin Ding, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. 2025. Geak: Introducing triton kernel ai agent & evaluation benchmarks. *Preprint*, arXiv:2507.23194.

Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. 2025. TritonRL: Training LLMs to think and code triton without cheating. *arXiv preprint arXiv:2510.17891*.

Ruiyuan Xu, Shuoming Zhang, Guangli Li, Qiuchu Yu, Rui Zhang, Yangyu Zhang, Hao Qian, Chunwei Xia, Jiacheng Zhao, Chenxi Wang, Xiaobing Feng, Jingling Xue, and Huimin Cui. 2026. LEGO: An LLM-enabled hierarchical optimizer for tensor computation graphs with structure-aware search and compositional synthesis. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. Code available at `https://github.com/mmt-at/LEGO`.

Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, Guang Liu, and Yonghua Lin. 2026. Towards automated kernel generation in the era of LLMs. *arXiv preprint arXiv:2601.15727*.

Genghan Zhang, Shaowei Zhu, Anjiang Wei, Zhenyu Song, Allen Nie, Zhen Jia, Nandita Vijaykumar, Yida Wang, and Kunle Olukotun. 2025a. Accelopt: A self-improving llm agentic system for ai accelerator kernel optimization. *Preprint*, arXiv:2511.15915.

Shuoming Zhang, Jiacheng Zhao, Qiuchu Yu, Chunwei Xia, Zheng Wang, Xiaobing Feng, and Huimin Cui. 2026. The new compiler stack: A survey on the synergy of LLMs and compilers. *Preprint*, arXiv:2601.02045.

Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. 2025b. Cudaforge: An agent framework with hardware feedback for cuda kernel optimization. *Preprint*, arXiv:2511.01884.

Xinguo Zhu, Shaohui Peng, Jiaming Guo, Yunji Chen, Qi Guo, Yuanbo Wen, Hang Qin, Ruizhi Chen, Qirui Zhou, Ke Gao, Yanjun Wu, Chen Zhao, and Ling Li. 2026. QiMeng-Kernel: Macro-thinking micro-coding paradigm for LLM-based high-performance GPU kernel generation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.

A Implementation of the Harness

A.1 Isolated scoring

We evaluate every kernel in its own subprocess. The subprocess loads the generated module, runs the correctness check and the timing loop, and writes the result as one JSON line on standard output. The parent process reads that line and collects the results. This design is required, not optional. The CUDA runtime keeps state per process, so if one bad kernel corrupts the context, then every later evaluation in the same process fails as well. Without isolation, a sequential run over 250 tasks is not meaningful.

A.2 Correctness check and chunked comparison

We draw two random inputs and accept the kernel when the absolute error against the reference stays below 10^{-2} on both (the framework's default path draws five, see Appendix M). A simple comparison allocates temporary tensors for the absolute difference, for the tolerance, and for the mask. Each of these has the size of the output, so the peak memory grows to three or four times the size of the output. On a 48GB GPU these temporary tensors alone made *correct* kernels fail with out of memory. To avoid this we compare in chunks of 64 million elements and we also collect diagnostic statistics during the comparison. We free the input tensors right after the forward pass, and we run the workers with an allocator option that reduces memory fragmentation. For speed we take the median of ten runs, and the speedup is the reference time divided by the generated time.

A.3 Prompt structure

The system prompt states the hardware facts of the target GPU. It gives the shared memory size, which is about 100KB or 101,376 bytes, the amount of device memory, the memory bandwidth, and the peak throughput. It also gives simple tile size rules, for example a block size of 64 or less for matrix multiplication and a block size of 1024 for element-wise work. We also inject a small database of Triton patterns and a set of architecture hints. The user prompt contains the full PyTorch reference and

the **input dtype contract**. The contract tells the model that it must not guess the dtype of the reference inputs and must not cast them. Without this contract the model writes a kernel that assumes fp16, and every such kernel fails on an fp32 reference. The repair prompt adds the previous code, the error message, and the index of the current round. We require the answer to be a single Python code block that contains a `Model` class derived from `torch.nn.Module` and a kernel function with the `@triton.jit` decorator.

A.4 Safety rules in the repair loop

The loop enters the optimization stage only after the kernel is correct. If the values do not match, we do not try to make the kernel faster. If the loop makes no progress for a fixed number of rounds, we throw away the accumulated context and the previous code, and we generate again from scratch. This second rule helps when the context is polluted, for example after an illegal memory access, but it does not help when the model simply cannot solve the task. In our runs the two cases were easy to tell apart. Tasks that a context reset rescued were different from tasks that failed even after two resets, and the second group was concentrated on three-dimensional and four-dimensional tensor products and on transposed matrix multiplication.

A.5 Response parsing and multiple APIs

The parser extracts the Markdown code block, renames the class to `Model`, and removes any prose that is not Triton code. This step absorbs the different output styles of different models. Commercial models use the chat completion API. Reasoning models do not accept a temperature parameter, so they use the response API instead. Open models use the completion API of a local vLLM server, and the quantized model uses llama.cpp. The client detects the right API from the model identifier, falls back when the first call fails, and caches the result so that it does not repeat the same detection on every call.

B Budget Settings and Wall-Clock Cost

The small budget is one sample and up to three repair rounds. We use it for full sweeps over 250 tasks. The large budget is three samples and up to nine repair rounds, which is at most 27 attempts per task. The multi-turn curve changes only the number of effective turns, from 1 up to 14.

A full sweep under the large budget is dominated by wall-clock cost. On a slow local server, 27 attempts per task did not fit into a two-hour limit. As a result 59 percent of the tasks did not finish, and the whole sweep took 22.9 hours. If we break down the result, 29 tasks were correct and 9 tasks were wrong after a normal evaluation, so only 38 tasks finished. The rest were 59 worker timeouts and 3 tasks with no result. One point is worth noting. Several tasks first looked like tasks that failed all 27 attempts, but in fact they were all timeouts and not wrong answers. A reader who only looks at the surface accuracy can easily read this as a limit of model ability.

We draw three design rules from this. First, set the worker timeout from the number of attempts times the average time per attempt, and not from a fixed guess. Second, keep the number of workers below the number of concurrent serving slots. Third, draw the samples of best-of-N in parallel and not one after another.

Parallelism at the task level was harmful. When we ran two tasks at the same time, the serving slots saturated, the throughput per slot dropped by more than half, generation exceeded the client timeout, and the retry loop repeated forever, so no task finished at all. Sequential execution was the better choice in our setting.

The repeated runs of Appendix M took 4.3 to 4.9 hours each on one RTX PRO 6000 that served the model and scored the kernels at the same time, about 32 GPU-hours in total. Every full Level 1 sweep under the small budget has this cost on this GPU, and the Level 2 and Level 3 sweeps take longer because the reference models are larger.

C Raw Data of the Repair-Round Ablation

Table 8 gives the full numbers behind Table 4 of the main text.

Table 8: Effect of the number of repair rounds on accuracy and on the error profile. Model is gpt-4o and the task set is KernelBench Level 1.

Rounds	Solved (/100)	Compile err.	Value mism.
1	19	59.3%	24.7%
3	21	51.9%	39.2%
5	19	45.7%	37.0%
10	22	32.1%	53.8%
Union	**28**	—	—

As the number of rounds grows, the share of compilation errors falls from 59.3 percent to 32.1 percent, while the share of value mismatches rises from 24.7 percent to 53.8 percent. Iterative repair fixes syntax errors well, but it does not fix semantic errors on its own. This matches the error transition finding of the main text. The union over the runs grows as 19, 25, 26, and 28, so it always stays above the best single run, which solved 22 tasks.

D Details of Model Cascading

The main text only reports the conclusion of the cascade experiment. Table 9 gives the accumulated result of each stage.

Table 9: Accumulated accuracy of the cascade. Each stage only runs on the tasks that the previous stage failed.

Configuration	L1	L2	L3	Total (/250)
gpt-4o-mini	15	1	0	16 (6.4%)
gpt-4o	19	6	0	25 (10.0%)
gpt-4o + gpt-5.2	42	15	4	61 (24.4%)
+ gpt-5.2-pro	43	15	4	62 (24.8%)
claude-opus-4-7 alone	46	–	–	–
opus + gpt-5.4-nano	48	–	–	–
union of five models	**51**	–	–	–

We observe three things. First, adding the next generation model to gpt-4o raises the total from 25 tasks to 61 tasks, which is a factor of 2.44. The 36 new tasks are 23 tasks on Level 1, 9 tasks on Level 2, and 4 tasks on Level 3. Second, the expensive reasoning model adds almost nothing. It ran on 225 tasks and added only one task, a Level 1 task with a speedup of 7.60 times. If we also consider its cost and its response time, its value inside a cascade is close to zero. Third, one recent flagship model alone, with 46 tasks, already beats the earlier three-model cascade with 43 tasks. The union of five models, with 51 tasks, still beats the best single model by 10.9 percent, so the value of independent samples does not disappear in the newest generation.

E Error Distribution by Level

Table 3 of the main text only covers Level 1. Table 10 covers all three levels. The timeout column counts our own response and evaluation time limits together with unclassified runtime errors. It is not a constraint of the benchmark.

Table 10: Share of each error type among failed tasks, by model and by level.

Model	Lv.	Failed	Compile	Value	CUDA	Timeout
gpt-4o-mini	L1	85	71.8%	20.0%	2.4%	2.4%
	L2	99	70.7%	22.2%	4.0%	1.0%
	L3	50	82.0%	6.0%	6.0%	0%
gpt-4o	L1	81	45.7%	37.0%	11.1%	0%
	L2	94	51.1%	41.5%	4.3%	0%
	L3	50	38.0%	56.0%	4.0%	0%
gpt-5.2	L1	58	6.9%	58.6%	1.7%	20.7%
	L2	85	25.9%	62.4%	0%	8.2%
	L3	46	17.4%	71.7%	0%	4.3%
gpt-5.5	L1	63	71%	3%	0%	25%
claude-opus-4-7	L1	54	0%	65%	0%	28%

The error transition appears at every level. For gpt-4o-mini the compilation error is dominant at all three levels, and for gpt-5.2 the value mismatch is dominant at all three levels. Inside one model the same shift also appears with difficulty. For gpt-4o the share of value mismatches grows from 37.0 percent on Level 1 to 56.0 percent on Level 3, so failures become more semantic as the task becomes harder.

For the reasoning model gpt-5.2-pro, 100 percent of the Level 1 failures and 81.9 percent of the Level 2 failures were timeouts. On Level 3 all failures came from API rate limits. That is a different cause, so we do not merge it into the timeout column.

F Details of the Speedup Results

We only measure speedup on kernels that pass the correctness check. Table 11 reports the result.

Table 11: Share of correct kernels that reach the Fast-1 and Fast-2 thresholds. The denominator is the number of correct kernels for that model.

Model	Fast-1	Fast-2
gpt-4o-mini	7/16 (43.8%)	1/16 (6.3%)
gpt-4o	12/25 (48.0%)	4/25 (16.0%)
gpt-5.2	25/36 (69.4%)	5/36 (13.9%)
gpt-5.2-pro	10/13 (76.9%)	3/13 (23.1%)
gpt-5.4-nano	21/35 (60.0%)	10/35 (28.6%)
gpt-5.5	22/37 (59.5%)	8/37 (21.6%)
claude-opus-4-7	29/46 (63.0%)	14/46 (30.4%)

The Fast-2 share grows from 6.3 percent to 30.4 percent across model generations, so newer models write not only more correct kernels but also better optimized ones. The reasoning model is an interesting exception. It solved only 13 tasks, which is the smallest number in the table, yet its Fast-2 share

is 23.1 percent, which is high. When a reasoning model does solve a task, it tends to write a well optimized kernel. Its overall success rate is low, however, so its practical value stays limited.

The largest single speedup that we observed was 84.03 times on one Level 3 task. Other high cases were 25.45 times, 19.10 times, and 11.86 times. These cases show that a generated Triton kernel can beat the default PyTorch implementation by a wide margin on some operation patterns. Speedup still depends on the hardware baseline, so these numbers are only comparable inside one hardware setting.

G Details of the Reused Open Artifacts

The main text only reports the result of each port. Here we describe how each artifact is built and what happened when we ran it.

G.1 Failure-rule memory

The system turns each failure into one short rule of the form "you must not do this", stores the rule together with a score, removes duplicates after normalization, and injects the top rules into the generation prompt. In one full evaluation the store collected between 73 and 80 distinct rules. Examples are a rule that says the kernel body must not call PyTorch operations, rules about the dimensions of a matrix multiplication and about the block size along the reduction axis, and rules about contiguity assumptions. The cost is one extra model call per failure, which is cheap, and this was the most portable artifact that we reused.

The risk, however, is not symmetric. A good rule saves a few attempts, while a wrong rule with a high score can block a whole class of tasks. In our runs the rules that the system collected while the evaluation harness was still broken polluted the store. The polluted store held 320 rules. Some of them treated an infrastructure defect as a rule for the model, for example a rule that said the GPU does not have enough memory to run the kernel. These rules had a high score and filled the top of the prompt. As a result the model failed on tasks as simple as a square matrix multiplication, even with nine repair rounds and three samples. Accuracy recovered after we cleared the store. There are two lessons. The store must be cleared after any infrastructure fix, and in a multi-server setup with a shared store the pollution spreads from one server to the other.

G.2 The Triton-specific reinforcement learning model

We served the released weights on a local vLLM server. This model does not support the system role of the chat format, so our first calls failed. We changed the client to fold the system message into the user prompt, and the calls then worked. On the first 20 tasks of Level 1 the model reached 0.85 on the 96GB GPU and 0.95 on the H100. The H100 result is higher because the model needs only about 16GB, so the scorer still had more than 55GB and no task failed with out of memory. Under the same conditions the 30B general open model reached only 0.60 on the H100. A small specialized model therefore beats a large general model and at the same time leaves more memory for the scorer, which matters in practice.

G.3 The open deep-agent framework

We cloned the released framework and ran it with our own open models. Three problems appeared during the port. First, the serving context length decides whether repair works at all. With a 16K context the repair prompt, which grows with the code and the error text, exceeded the input and output limit, and every round was lost. With a 64K context the same tasks were solved in a few tens of seconds. Second, installing an external project into a shared environment breaks the dependencies. The editable install of the framework raised the version of shared packages, and the serving engine then failed to start. An external project must be installed into its own virtual environment. Third, the shell command returned an exit code of zero even when the HTTP response was an error, and we twice concluded that the server was healthy when it was not. A health check must look at the status code of a real completion call.

We also note that this framework verifies its answers with tests that the model itself writes, and its tolerance is looser than ours. Its accuracy numbers and our accuracy numbers therefore use different verification strengths, which is one more reason why numbers from different systems should not be compared directly.

G.4 The multi-turn reinforcement learning environment

We reproduced the released environment on the 100 tasks of Level 2. Table 12 gives the metrics that the main text does not list.

Table 12: Reproduction of the released multi-turn reinforcement learning environment on KernelBench Level 2 with 100 tasks. All values are computed over all tasks.

Metric	Original paper	Our reproduction
Fast@1	49.2%	42.9%
Fast@1.2	25.6%	24.3%
Fast@1.5	7.4%	12.9%
Fast@2	2.1%	7.5%

Fast@1.2 is almost the same and Fast@1.5 and Fast@2 are higher, so we judge the reproduction to be successful. Fast@1 is 6 points lower, which we attribute to a different GPU and to sampling noise. Correctness alone was 76.3 percent, and pass@1 under a loose threshold was 85.7 percent.

The infrastructure cost was high. The grading path of the trainer needs the FSDP actor and the vLLM rollout engine in the same memory, and five attempts on a single 96GB GPU all failed. An eight-GPU node was in practice the minimum requirement. We also had to clear the existing cluster completely before each run, because leftover processes of a different version attached themselves to our job. The released validation data covers Level 2 only, so the same reproduction on Level 1 and Level 3 would need a new data set in the same format. The point of this subsection is that reproducibility does not imply accessibility.

H Details of the Quantization Study

We loaded a very large open reasoning model with aggressive quantization and evaluated it under the same protocol. We served it with layer-split pipeline parallelism and we kept one separate GPU for scoring.

The 2-bit version was about 254GB. It reached 0.10 on Level 1 and solved nothing at all on Level 2, on Level 3, and on the real serving kernels. It needed about 8.5 hours per level and 21.5 hours in total. It spent about 2,600 tokens of thinking per attempt with no gain in accuracy. A general reasoning model with two orders of magnitude more parameters therefore stayed below half of the 8B specialized model.

The 4-bit version was about 467GB, which was the highest precision that fit into our serving budget. Accuracy recovered to 0.96 over the tasks that finished inside the time limit, which confirms that quantization loss was the cause of the collapse. This version, however, generated only about 8 tokens per second. As a result 76 percent of the tasks exceeded the two-hour limit, the surface accuracy was only 0.23, and the run took 30 hours.

Taken together, under a fixed GPU budget a very large model at low precision loses quality, and the same model at full precision loses speed. A mid-size model at full precision is the better choice for kernel generation.

I Details of the Cross-Benchmark Study

I.1 The Triton-specific benchmark

This benchmark has 166 tasks and uses a different evaluation style from KernelBench. Each task is a PyTorch function together with a test harness, so a scorer that expects a module and an input generator cannot run it. We therefore wrote a separate runner. The runner executes the reference and the generated code with the same test harness, compares the two result dictionaries, and reports both whether the code runs and whether the results match. Over the four combinations of two models and two GPUs, execution accuracy stayed inside a narrow band between 0.151 and 0.187, and the advantage of the specialized model that we saw on KernelBench disappeared. The suite also contains tasks such as optimizer updates, which do not map naturally onto a Triton kernel, so a part of the suite is structurally out of reach.

To find the cause of the gap against the original paper, we ran a two-by-two ablation over the prompt format and the budget. Aligning the prompt with the training format of the specialized model alone did not help, and execution accuracy even fell from 0.151 to 0.127. Turning on the budget raised execution accuracy by about 2.3 times and call accuracy by about 2.5 times. Under the larger budget the conversion-style prompt, which provides the PyTorch reference, was in fact better than the aligned prompt. The gap is therefore explained by single-attempt evaluation and not by a mismatch with the training prompt.

I.2 The standardized agent environment

This environment has 139 standard tasks and 13 out-of-distribution tasks. Under a single attempt, 106 of the 139 standard tasks compiled, and 17 tasks passed both compilation and the correctness check, which is 12.2 percent. The geometric mean of the performance of the passing kernels relative to the oracle was 0.143, so they were about seven times slower than the oracle, and only three kernels

were faster than the oracle. The main cause of compilation failure was an import error, with 29 cases in which the model used a module or a symbol that does not exist. On the 13 out-of-distribution tasks, 12 tasks compiled but only one task passed, which is 7.7 percent.

With a test-time scaling agent, Pass@1 on the standard tasks rose to 18.0 percent and the geometric mean of performance rose to 0.285, which is about twice as high. The compilation rate, however, fell to 45.3 percent, because the last proposal of a repair loop sometimes does not compile. On the out-of-distribution tasks Pass@1 stayed at 7.7 percent and did not move at all. This is the measurement behind the boundary result of the main text.

I.3 The track for a new domain-specific language

The same environment has a track with 12 tasks for a new domain-specific language. Nine of these tasks could not run in our environment at all. Some of them need a separate compiler fork, and others use an API that only exists in a newer version. On the three tasks for which the oracle could run, the specialized reinforcement learning model passed none under a single attempt. The sample is small, so we do not draw a quantitative conclusion. We report it as evidence that the collapse outside the training distribution is not limited to one axis of one benchmark.

I.4 The real serving kernel benchmark

We took 8 dense tasks from a benchmark of real serving kernels for a recent NVIDIA architecture and converted them into the KernelBench format. The conversion turns the reference definition and the tensor specification into a module and an input generator. Among the four combinations, only the specialized model on the H100 reached 0.25, and the other three stayed at zero. This points to differences in the Triton version and in the compute architecture. The attention and mixture-of-expert tasks need structured input synthesis, so we leave them for future work.

J Beyond the GPU: Two NPU Targets

To test whether the findings are properties of GPUs and Triton or of the generate-and-verify loop itself, we ported the harness, unchanged in its structure, to two neural processing units with different programming models. The first exposes a Triton dialect, so the language is familiar to the models. The second exposes a Rust-embedded domain-specific language that no model has seen at scale. Both experiments use the same A/B design as the rule-memory study: condition A gives the model the vendor documentation alone, and condition B adds one page of constraints that we measured on the device.

Table 13: Correct kernels out of 15 (five operators, three samples) on the Triton-dialect NPU. Condition A gives the vendor documentation; condition B adds one page of measured constraints. No repair round was ever triggered, because every failing kernel compiled.

Model	A	B
claude-opus-4-8	12/15	15/15
claude-sonnet-5	12/15	15/15
gpt-5.5	12/15	15/15
gemini-3.1-pro-preview	12/15	15/15
deepseek-v4-pro	10/15	15/15
qwen3.8-max	8/15	13/15
all models	66/90	88/90
of which GELU	0/18	18/18

J.1 A Triton dialect on a commercial NPU

The first target is a data-center NPU whose SDK accepts Triton kernels through a vendor compiler. We took five operators that a transformer block needs, namely GELU, layer normalization, matrix multiplication, attention, and a block-scaled product, and asked six current gateway models for three samples each, which gives 90 candidates per condition. Every candidate passes the compile gate on the device host and is then executed on the NPU against the PyTorch reference, with the signal-to-quantization-noise ratio as the correctness measure and the median of three latency runs.

Table 13 shows the result. Under condition A, 85 of the 90 candidates compile but only 66 are correct, and the gap has one dominant cause. Sixteen of the 24 failures are kernels that compile, run, and return uninitialized memory. The vendor's supported-operation list includes the sigmoid, the error function, the minimum, and the conditional select, but none of them lowers on the device, and a Python scalar literal inside a tensor expression does not lower either. When lowering fails, the runtime silently falls back and the output buffer is never written. GELU needs exactly these operations, so all 18 GELU candidates compile and all 18 are wrong, for every model. The compile

Table 14: The 30B open coder on three NPU kernels in a Rust-embedded DSL, small budget. A is the vendor documentation and B adds the measured constraints. A candidate counts as an improvement only if it compiles, changes the static schedule, and shortens it. The last column counts contract violations.

Cond.	Compiled	No-op	Improved	Violations
A	7/9	7	0	1
B	7/9	6	0	3

gate is blind to this failure, and so is any report that counts compiled kernels. Only execution with a numerical comparison catches it, which is the verification-strength axis of Finding 2 on a second platform.

Condition B adds one page that lists which operations really lower, how to build a constant from a tensor, and how to compose a sigmoid from the exponential. With this page the five flagship models reach 15 of 15 and the total rises from 66 to 88 of 90; the two remaining failures are gateway timeouts. The generated kernels run at the latency of the hand-written reference, with a median ratio of 0.99 in both conditions. A local 8B open model under the same setup solved only the matrix multiplication, one of three operators we tried, so the result above is a property of the flagship generation. Measured platform knowledge, then, transfers completely when the model already writes the language fluently, and the number that moved was not the compile rate but the share of compiled kernels that are correct.

J.2 A Rust-embedded DSL on a second NPU

The second target shares nothing with Triton: three attention and feed-forward kernels of a 12B decoder, written in a Rust-embedded domain-specific language for a different NPU, taken from a public kernel-optimization challenge. The harness adds a contract gate that rejects edits outside the permitted files or to any device-function signature, and the compile gate classifies the result into a Rust front-end error, a backend error, or a compiling kernel with a static schedule length. Because a compiling kernel is not an improved kernel, two no-op detectors follow the compile gate: identity of the source after comment removal, and identity of the static schedule after source positions are stripped. We ran the 30B open coder with three samples and three repair rounds per kernel under both conditions.

Table 14 shows the result over 42 attempts and about 400K output tokens. Fourteen of eighteen samples compiled, and thirteen of the fourteen were no-ops: the model added comments, renamed a helper, or undid its own edit in the repair round to satisfy the compiler. The one candidate that changed the schedule left its length unchanged. Over all attempts the Rust front end rejected 57 percent, and about four fifths of those rejections were one grammar rule, a mapping macro used in a type position. No candidate reached the backend compiler. Here the measured constraints did not help. The page describes what to do, the model could not express it in the language, and the contract violations rose from one to three. The hand-optimized kernels on the same tasks run about seven times faster than the baseline, and the model recovered none of that gap.

The two targets together sharpen Finding 8. The same page of measured knowledge moved accuracy from 66 to 88 of 90 where the model already spoke the language and moved nothing where it did not. Budget and knowledge act after the syntax wall, not before it, and the stage at which a target fails decides what any further investment can buy.

K Details of the Cross-Hardware Study

Under the same conditions the 30B open model reached 0.75 on the 96GB GPU and 0.60 on the H100 over the first 20 tasks of Level 1. The main cause is GPU memory. The H100 has 80GB, so after serving the 30B model the scorer had only about 43GB, while the 96GB GPU left about 52GB. For tasks with large inputs this difference alone decides whether the task fails with out of memory. If we correct for the out-of-memory cases, the H100 number rises to about 0.70, which is close to the 96GB GPU. We also tried to lower the serving share further, but then the server did not start at all because the key-value cache became too small, so this environment has a lower bound.

Speedup behaves in the opposite way and depends strongly on the hardware. On a square matrix multiplication we measured 1.61 times on the 96GB GPU and 0.83 times on the H100. The cuBLAS baseline of the H100 is very strong, so a generated Triton kernel is at a disadvantage there. The median speedup was 1.51 against 1.44, and the Fast-2 share was 0.10 against 0.05.

We repeated the run on a third GPU with 48GB and obtained a similar profile across levels, with 0.37 on Level 1, 0.12 on Level 2, and 0.04 on

Level 3. This supports the conclusion that accuracy is almost independent of the GPU when memory is sufficient. This GPU, however, produced only about one fourth to one fifth of the throughput of the H100, so the same sweep took about three times longer in wall-clock time. Accuracy is independent of the hardware, but whether the experiment can be run at all is not.

L Practical Notes for Reproduction

Several operational factors distorted our measurements more than once, and we list them here because they are easy to miss and hard to detect from the accuracy number alone.

First, serving and scoring must not compete for the same memory. If the serving process takes too large a share of the GPU, even a trivial kernel fails, as Finding 2 shows. Second, a released model artifact can carry its own generation limit, and the server clamps the client limit to that value, so raising the limit on the client side alone does not remove truncation. Third, a reasoning model spends thousands of tokens before it writes code, so the common response limit of the agent framework is too small and cuts the answer before the code block. Fourth, the serving context length must hold the repair prompt, which grows with the code and the error text, otherwise every repair round is lost. Fifth, an experience memory that was filled while the harness was still broken keeps poisoning later runs, so the store must be cleared after an infrastructure fix. Sixth, a shared store spreads that damage across servers, so each experiment should use its own path.

We also found that parallelism at the task level can be harmful. When we ran two tasks at the same time, the serving slots saturated, the throughput per slot dropped, generation exceeded the client timeout, and the retry loop never terminated, so no task finished at all. Sequential execution was the better choice on our hardware.

M Run-to-Run Variance and Harness Sensitivity

All numbers in the main text are single runs except the Level 1 accuracy of the 8B open model under the small budget, which we repeated to measure how much one run of this benchmark moves. Table 15 lists the runs. The sampling temperature is 0.2, so the variance comes from sampling in generation and repair, not from the scorer, which is deterministic for a given kernel.

Table 15: Repeated runs of the 8B open model on all 100 tasks of Level 1 under the small budget. The first row uses the harness of the main text (runs: 0.47, 0.44, 0.39, 0.47) and the second row uses the framework's own command-line path (runs: 0.31, 0.41, 0.33). Union and intersection count the tasks solved by any run and by every run of a row.

Path	Runs	Mean $\pm$ sd	Union	Inter.
main-text harness	4	0.44 ± 0.04	59	25
command-line path	3	0.35 ± 0.05	50	23

Two observations follow. First, the standard deviation of a single full sweep is four to five points. Any difference of that size between two rows of a table in this paper is within noise, and only the gaps between model generations, which are ten points and more, are safe to interpret. Second, the set of solved tasks moves more than the count. The four runs of the main-text harness agree on only 25 tasks, yet together they cover 59, which is twelve more than the best single run. This is the same effect that Finding 4 reports for repair depth, and it appears here without any change of budget. Breadth, in the form of independent runs, buys coverage that depth does not.

The second block is a harness effect rather than a model effect. Both paths load the same model, the same prompt templates, and the same task set with one sample and three repair rounds. They differ in five settings. The main-text harness runs one operating-system process per task in sequence, waits up to 2400 seconds per task, samples at temperature 0.2, verifies on two random inputs, and keeps profiling off. The command-line path runs two tasks in parallel threads, stops each evaluation after 120 seconds, samples at temperature 0.3, verifies on five random inputs, and runs the profiler. None of these settings is part of the model, and together they move the mean by nine points, which is more than twice the run-to-run standard deviation. This is why we report the settings of Section 3 with every number and why Finding 2 lists them among the axes that an accuracy report must state.

N Hidden Variables and a Reproduction Checklist

We list the factors that distorted our measurements more than once. They have one thing in common. None of them appears in an accuracy report.

- **Keep serving memory and scoring memory apart.** If the serving process takes too large a share of the GPU, even a trivial kernel fails with out of memory. When serving and scoring share one GPU, the serving share must match what the scorer needs.
- **Check the generation limit inside the model artifact.** The maximum generation length in the released generation config is clamped by the server. Raising the limit on the client side alone does not remove truncation, so the artifact itself must be checked.
- **Raise the response token limit of the framework.** A reasoning model spends thousands of tokens on its thinking trace before it writes any code. A common limit of 8K cuts the answer before the code block and turns a solvable task into a task with no code. In our runs, raising the limit to 13,000 moved accuracy on the affected range from 0.37 to 0.78.
- **Make the serving context long enough.** The repair prompt grows with the code and with the error text. If the context is short, every repair round is lost.
- **Clear the experience memory after an infrastructure fix.** Rules that were collected while the harness was broken keep lowering accuracy in a healthy environment.
- **Give each experiment its own store path.** When several servers share one experience store, the pollution spreads from one experiment to the next.
- **Prefer sequential execution on a small serving cluster.** If the serving slots saturate, the throughput drops and the retry loop does not terminate.

Finally, we repeat the axes that should be reported next to any accuracy number. These are the attempt budget, the verification strength, the memory headroom of the scorer, the serving context length, the response token caps of both the framework and the model artifact, and the evaluation distribution. A comparison of numbers without these axes is a comparison of measurements from different regimes.